

GEMM Working Note #1

deval-d@stanford.edu

May 30, 2026

While working on GEMM, I'm making small notes to document implementation ideas as I go along. After I'm satisfied with `1ak`'s performance, I'll consolidate them into a comprehensive note.

Consider the matrix product $AB = C$, where A is $m \times k$, B is $k \times n$, and C is $m \times n$.

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{bmatrix} \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \\ b_{30} & b_{31} & b_{32} \end{bmatrix} = \begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix} \quad (1)$$

Then, the first column of C , for example, is given by

$$c_{:,0} = b_{00}a_{:,0} + b_{10}a_{:,1} + b_{20}a_{:,2} + b_{30}a_{:,3} \quad (2)$$

Equivalently, for general C column j ,

$$c_{:,j} = b_{0,j}a_{:,0} + b_{1,j}a_{:,1} + b_{2,j}a_{:,2} + b_{3,j}a_{:,3}. \quad (3)$$

1 Contiguity and Fusing

`1ak` is contiguous-only and column-major, so the natural unit of work is a column slice. In GEMM, this matters because every column of C is formed from the same columns of A . For the example above,

$$\begin{aligned} c_{:,0} &= b_{00} \mathbf{a}_{:,0} + b_{10} \mathbf{a}_{:,1} + b_{20} \mathbf{a}_{:,2} + b_{30} \mathbf{a}_{:,3} \\ c_{:,1} &= b_{01} \mathbf{a}_{:,0} + b_{11} \mathbf{a}_{:,1} + b_{21} \mathbf{a}_{:,2} + b_{31} \mathbf{a}_{:,3} \\ c_{:,2} &= b_{02} \mathbf{a}_{:,0} + b_{12} \mathbf{a}_{:,1} + b_{22} \mathbf{a}_{:,2} + b_{32} \mathbf{a}_{:,3}. \end{aligned}$$

The scalar weights from B change from one column of C to the next, but the columns of A do not. This suggests that a GEMM kernel should not finish one column of C at a time. Instead, it should hold several columns of C in registers, stream through the shared columns of A , and use the corresponding scalars from B to update all of those C columns before storing them back. In other words, the kernel should look like a fused version of the GEMV/`faxpy` idea. Rather than repeatedly loading and storing the same output column, we keep a small panel of C live, reuse the same column slices of A , and apply different scalars (contiguous in memory) from B to update multiple columns of C at once. This has two main benefits:

1. Each active column of C is loaded once, updated many times, and stored once.
2. The same entries of A are reused across several columns of C while they are still hot.

The payoff is the same as in the fused GEMV `faxpy` kernel, but stronger: GEMM has enough reuse that the kernel can amortize memory traffic much more effectively. The only tricky part is how to cleanly architect this routine. There are multiple levels of loops and blocking and tail-handling that I need to deal with.

2 Rough Kernel Blocking Idea

Suppose I want to update the *block* of C below. Blocks are used instead of full column-panels to be able to fit all the microkernel work in nearby caches.

$$\begin{bmatrix} \boxed{\begin{matrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{matrix}} & c_{02} \\ c_{20} & c_{21} & c_{22} \end{bmatrix}$$

Here, this block is square 2×2 . But consider it being $\text{MC} \times \text{NC}$ in general. Then, we need to do the following updates:

$$\begin{aligned} c_{0..2,0} &= b_{00}a_{0..2,0} + b_{10}a_{0..2,1} + b_{20}a_{0..2,2} + b_{30}a_{0..2,3} \\ c_{0..2,1} &= b_{01}a_{0..2,0} + b_{11}a_{0..2,1} + b_{21}a_{0..2,2} + b_{31}a_{0..2,3} \end{aligned}$$

Therefore, for an $\text{MC} \times \text{NC}$ block of C , we use a full $\text{MC} \times k$ panel of A and values from a full $k \times \text{NC}$ panel of B .

$$\begin{bmatrix} \boxed{\begin{matrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{matrix}} \end{bmatrix} \begin{bmatrix} \boxed{\begin{matrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \end{matrix}} & \begin{matrix} b_{02} \\ b_{12} \\ b_{22} \\ b_{32} \end{matrix} \end{bmatrix} = \begin{bmatrix} \boxed{\begin{matrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{matrix}} & c_{02} \\ c_{20} & c_{21} & c_{22}, \end{bmatrix} \quad (4)$$

However, a full $\text{MC} \times k$ panel of A and a full $k \times \text{NC}$ panel of B may be too large to all keep in cache. We could make MC and NC smaller until these panels fit, but then less of C is updated at once. Hence, fewer columns of C can be fused together to share the same work from A . This leads to more loads and stores of C , which defeats the point of fusing in the first place. Instead, the operation is further split along the inner k dimension. We iterate over an $\text{MC} \times \text{KC}$ panel of A and a $\text{KC} \times \text{NC}$ panel of B at a time, with each pair of panels contributing a partial update to the same $\text{MC} \times \text{NC}$ block of C . My head is starting to get dizzy with all these loops to handle.

3 Regimes

For small matrices, the overhead introduced by blocking by MC is not needed and costly. For these matrices, I'll fuse full column panels of C together instead. Hence, the small matrix kernel will fuse an $m \times \text{NC}$ column panel of C , which involves operating on many $m \times \text{KC}$ panels of A and $\text{KC} \times \text{NC}$ panels of B . Afterwards, for larger matrices, the macrokernel will switch to use the $\text{MC}/\text{KC}/\text{NC}$ variant instead to amortize memory more effectively. I'm not entirely sure when this transition occurs on Apple Silicon. Therefore, I'll have to tune the small matrix kernel cutoff depending on m , k , n matrix dimensions.

Although, because the only loop in the small matrix kernel not present is MC blocking, the transition will mostly depend on m , I think. Once the C matrix gets fairly “tall,” that's when blocking by MC as well will start to pay off. So the “small matrix kernel” isn't really for small matrices, in a sense. Large “wide” matrices shaped

$$(32, 4096) \times (4096, 32) \rightarrow (32, 32),$$

would still use the smaller kernel, as the height is only 32. But for taller (larger m) C matrices, it would transition to blocking by MC .

In summary, `1ak`'s GEMM will involve two kernel variants. A “short” kernel for C matrices of small row dimension m , and a “tall” kernel for C matrices of large row dimension m . The short kernel will fuse NC full column panels of C , whereas the tall kernel will block $\text{MC} \times \text{NC}$ of C at a time. Both kernels will involve blocking by NC/KC in A and B . The only difference is A is partitioned with full row panels $m \times \text{KC}$ for the short kernel, and partitioned into blocks $\text{MC} \times \text{KC}$ for the tall kernel.